DECK 2 OF 13 · FRC JAVA CRASH COURSE

KitBot drive: your first command-based robot

From an empty project to a 2026 KitBot you drive with a gamepad

WPILib 2026 · Java · Command-based

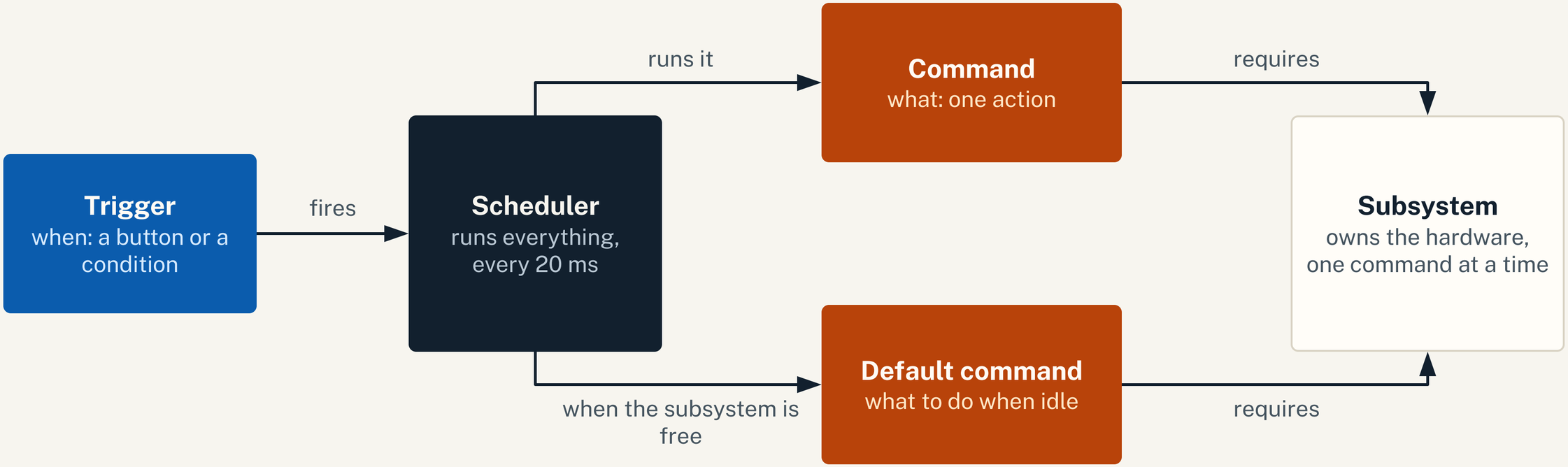
OVERVIEW

What this deck covers

- What a **subsystem**, a **command**, a **trigger**, a **default command** and the **scheduler** each do.
- CANDriveSubsystem for the four CIM motors on the KitBot.
- Binding a gamepad, and a safe first test with the wheels off the ground.
- Five common bring-up mistakes and their symptoms.
- Commands that run one after another, and a timed autonomous routine.
- Open loop, and why it works with a human driver.

COMMAND-BASED

The five parts of a command-based robot



Schematic. The scheduler is the only part that runs anything.

2027: Commands v3 is new for Java. The v2 you learn here remains.

PROJECT LAYOUT

The files in a command-based project

File	Its job	Keep in mind
Robot.java	Creates RobotContainer and runs the scheduler every loop	Keep it mostly empty
RobotContainer.java	Creates subsystems and controllers, and binds them to commands	Most of your setup goes here
Constants.java	Gives every number a name, grouped by subsystem	No logic in this file
subsystems/CANDriveSubsystem.java	Owns the motors and offers commands	Nothing else touches the motors

PROJECT SETUP

Creating the project and installing REVLib

1. **Open the WPILib command palette**
   Press Ctrl+Shift+P and choose WPILib: Create a new project.

2. **Choose Template, Java, Command Robot**
   The plain one, not the Skeleton: its comments are worth reading.

3. **Set the folder, name and team number**
   Keep it out of OneDrive, then click **Generate Project**.

4. **Install REVLib**
   Manage Vendor Libraries, then Install new libraries (online), then paste https://software-metadata.revrobotics.com/REVLib-2026.json

5. **Build once**
   Run WPILib: Build Robot Code to prove the library is in.

6. **Set up each SPARK in REV Hardware Client 2**
   Use 2026 firmware (26.x), then set the CAN ID and motor type.

REVLib install

docs.revrobotics.com: Installation

CANDRIVESUBSYSTEM

Step 1: create the motors

CANDriveSubsystem.java

```
  // CIMs are BRUSHED motors: the classic KitBot bug.
2 private final SparkMax leftLeader =
1     new SparkMax(LEFT_LEADER_ID, MotorType.kBrushed);
2 private final SparkMax leftFollower =
1     new SparkMax(LEFT_FOLLOWER_ID, MotorType.kBrushed);
2 private final SparkMax rightLeader =
1     new SparkMax(RIGHT_LEADER_ID, MotorType.kBrushed);
2 private final SparkMax rightFollower =
1     new SparkMax(RIGHT_FOLLOWER_ID, MotorType.kBrushed);

3 private final DifferentialDrive drive =
3     new DifferentialDrive(leftLeader, rightLeader);
```

1. **Brushed**
   CIMs have brushes. Tell the SPARK, because it can’t detect that.

2. **Private**
   Only this class touches the motors.

3. **Leaders only**
   DifferentialDrive gets one motor per side. Followers copy them.

CANDRIVESUBSYSTEM

Step 2: build the config

CANDriveSubsystem.java

```
  SparkMaxConfig common = new SparkMaxConfig();
1 common.smartCurrentLimit(CURRENT_LIMIT_AMPS);
1 common.idleMode(IdleMode.kBrake);

  // Leaders only: followers copy their leader's output.
  SparkMaxConfig rightConfig = new SparkMaxConfig();
2 rightConfig.apply(common).voltageCompensation(NOMINAL_VOLTS);
  // One side must be inverted. On the KitBot, it's the left.
  SparkMaxConfig leftConfig = new SparkMaxConfig();
3 leftConfig.apply(rightConfig).inverted(true);

  SparkMaxConfig leftFollowerConfig = new SparkMaxConfig();
4 leftFollowerConfig.apply(common).follow(leftLeader);
  SparkMaxConfig rightFollowerConfig = new SparkMaxConfig();
4 rightFollowerConfig.apply(common).follow(rightLeader);
```

1. **Shared by all four**
   The current limit and brake mode.

2. **Leaders only**
   Voltage compensation. A follower mirrors the result, so it needs none.

3. **One side inverted**
   The sides face opposite ways, and nothing flips one for you.

4. **Followers**
   Each one mirrors its leader. Set nothing else on direction.

CANDRIVESUBSYSTEM

Step 3: apply the config

CANDriveSubsystem.java

```
  // Reset: a swapped-in SPARK starts from a known state.
  // Persist: settings survive a brownout or breaker trip.
1 ResetMode reset = ResetMode.kResetSafeParameters;
2 PersistMode persist = PersistMode.kPersistParameters;
3 leftLeader.configure(leftConfig, reset, persist);
3 rightLeader.configure(rightConfig, reset, persist);
3 leftFollower.configure(leftFollowerConfig, reset, persist);
3 rightFollower.configure(rightFollowerConfig, reset, persist);
```

1. **Reset**
   Starts from a known state, so a swapped-in spare behaves.

2. **Persist**
   Saves to the SPARK, so settings survive a brownout or breaker trip.

3. **Once, at startup**
   Saving takes time and blocks the SPARK. Never do it in a loop.

CANDRIVESUBSYSTEM

Step 4: write the command factory

CANDriveSubsystem.java

```
/** Suppliers, not doubles: the stick is re-read on every 20 ms loop. */
public Command driveArcade(DoubleSupplier forward, DoubleSupplier rotation) {
  return run(() -> drive.arcadeDrive(forward.getAsDouble(), rotation.getAsDouble()))
      .withName("Arcade Drive");
}
```

run() makes a command that requires this subsystem. **Suppliers** let it re-read the stick on every loop.

ROBOTCONTAINER

Step 5: bind the controller

RobotContainer.java

```
private void configureBindings() {
  // Sticks read negative when pushed away, and +rotation is counter-clockwise.
  drive.setDefaultCommand(
      drive.driveArcade(
          () -> -MathUtil.applyDeadband(driver.getLeftY(), DEADBAND) * DRIVE_SCALE,
          () -> -MathUtil.applyDeadband(driver.getRightX(), DEADBAND) * TURN_SCALE));
}
```

Minus signs: match the stick to the robot. **Deadband:** ignore a stick that rests slightly off zero. **Scale:** reduce the speed.

FIRST TEST

First test: wheels off the ground

1. **Prop the robot up**
   Wheels clear, hands clear. One person ready on Enter (disable). Space is E-stop.

2. **Read the SPARK lights**
   Blue or yellow means brushed. Our code sets brake, so expect blue.

3. **Check the controller**
   Driver Station USB Devices tab: slot 0, and the axes follow the sticks.

4. **Enable Teleop and push forward gently**
   Both sides spin forward at the same speed.

5. **Push the right stick right**
   Left wheels go forward, right wheels go back.

6. **Let go, then check**
   Wheels stop. No red text in Messages. Nothing hot, and no bad smells.

COMMON MISTAKE

CIMs set to kBrushless

What you see	The SPARK light shows magenta or cyan, the brushless colours, and may blink orange and magenta for a sensor fault.
Why it happens	A new SPARK ships set to brushless and can’t detect the motor. In that mode it looks for a motor sensor a CIM doesn’t have.
The fix	Use MotorType.kBrushed in code, and set the motor type to brushed in REV Hardware Client 2. Look for blue or yellow.

COMMON MISTAKE

A CAN ID that does not match

What you see	The log shows [Spark Max] IDs: 6, Unable to retrieve SPARK firmware version and timed out while waiting. That motor stays still.
Why it happens	No SPARK answers at the ID in Constants. Every new SPARK ships as ID 0, and each needs its own number.
The fix	Set IDs in REV Hardware Client 2 over USB, robot off, one SPARK at a time. Then match Constants.

COMMON MISTAKES

Three more symptoms and what causes them

What you see	Why it happens	The fix
Push forward and the robot spins on the spot	No side is inverted, so the two sides drive opposite ways	Add inverted(true) on one leader. The KitBot uses the left.
One side is weak, or motors run hot and a breaker trips	A follower is missing, follows the wrong leader, or is reversed, so two CIMs fight in one gearbox	Give each follower .follow(itsLeader), with no true.
The left stick does nothing and the D-pad drives	The F310 Mode light is on, or the back switch is on D	Switch to X, Mode light off, replug. Check the USB Devices tab.

BEST PRACTICE

Never make the loop wait

> **No while loops or Thread.sleep in robot code.**

Everything shares one 20 ms loop. If one command waits, every subsystem waits, and motor safety stops the drive.

```
// Wrong: the whole robot freezes for 2 seconds
Thread.sleep(2000);

// Right: the scheduler keeps looping while this runs
drive.driveArcade(() -> 0.5, () -> 0).withTimeout(2.0);
```

SEQUENCES

One command that does three things in order

DriveAutos.java

```java
/** Drive forward, hold still, then turn left: one command that does three things. */
public static Command forwardPauseTurn(CANDriveSubsystem drive) {
  return drive.driveArcade(() -> AUTO_SPEED, () -> 0.0).withTimeout(DRIVE_OUT_SECONDS)
      .andThen(drive.driveArcade(() -> 0.0, () -> 0.0).withTimeout(PAUSE_SECONDS))
      .andThen(drive.driveArcade(() -> 0.0, () -> AUTO_SPEED).withTimeout(TURN_SECONDS));
}
```

withTimeout ends each step. andThen starts the next step when the one before it ends.

COMMON MISTAKE

A waitSeconds step in a drive sequence

What you see	During the pause, the Driver Station shows DifferentialDrive... Output not updated often enough.
Why it happens	Commands.waitSeconds runs no drive code, so nothing updates the drive. After 0.1 s, motor safety stops it and reports the error.
The fix	Hold the drive still with a step that drives at zero for the pause, as on the slide before.

```
// Error during the pause
forward.andThen(Commands.waitSeconds(1.0)).andThen(turn);
```

SEQUENCES

Ways to combine commands

Write this	It runs	It ends when
a.andThen(b)	a, then b	b ends
Commands.sequence(a, b, c)	each one in order	the last one ends
a.alongWith(b)	a and b at the same time	both have ended
a.raceWith(b)	a and b at the same time	either one ends
a.deadlineFor(b)	a and b at the same time	a ends, and b is cancelled
a.withTimeout(2.0)	a for at most 2 seconds	a ends or the time is up
a.until(condition)	a	a ends or the condition is true

AUTONOMOUS

An autonomous routine is one command

DriveAutos.java

```java
/** The same routine as a list: easier to read, reorder and extend. */
public static Command leaveAndTurn(CANDriveSubsystem drive) {
  return Commands.sequence(
          drive.driveArcade(() -> AUTO_SPEED, () -> 0.0).withTimeout(DRIVE_OUT_SECONDS),
          drive.driveArcade(() -> 0.0, () -> 0.0).withTimeout(PAUSE_SECONDS),
          drive.driveArcade(() -> 0.0, () -> AUTO_SPEED).withTimeout(TURN_SECONDS),
          Commands.print("Auto finished"))
      .withName("Leave and turn");
}
```

getAutonomousCommand() in RobotContainer returns this. Robot.java runs it when autonomous starts and cancels it when teleop starts.

BEST PRACTICE

Build a new command for each use

> **Call the factory again each time. One command object can go in only one group.**

Reusing one makes WPILib throw: Commands that have been composed may not be added to another composition or scheduled individually!

```
// Wrong: one command object in two groups
Command fwd = drive.driveArcade(() -> 0.4, () -> 0.0).withTimeout(2.0);
Command autoA = Commands.sequence(fwd, Commands.print("A"));
Command autoB = Commands.sequence(fwd, Commands.print("B")); // throws
// Right: each call to the method makes a new command
Command autoC = DriveAutos.leaveAndTurn(drive);
```

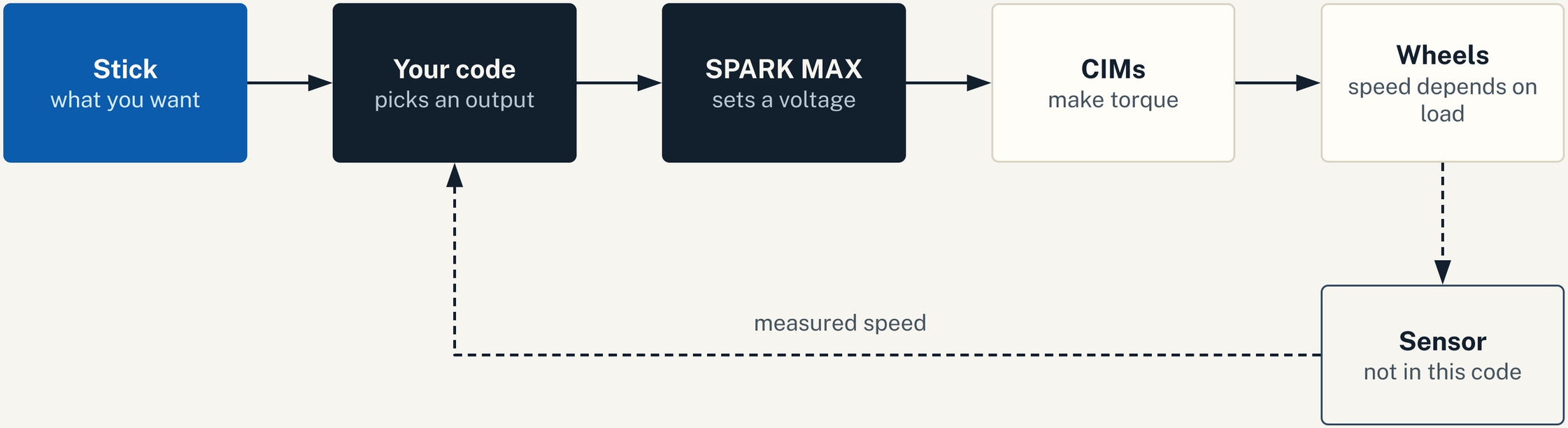
BEST PRACTICE

Five habits that keep robot code readable

Habit	In practice	Why
Constants, not magic numbers	LEFT_LEADER_ID, not 1	Swap a motor and change one line
The subsystem owns its hardware	Motors are private	Every use goes through the subsystem
Factories, not loose classes	drive.driveArcade(...)	Less code, and the requirement is automatic
Name your commands	.withName(“Arcade Drive”)	Logs show a name you can read
Commit to git often	Commit after each thing works	You can always get back to working code

CONTROL THEORY

Open loop: voltage with no measurement



Schematic. Solid arrows are this robot. Dashed arrows are what closed loop adds, in Deck 6.

EXERCISE

Change the drive scale

1. Set DRIVE_SCALE to 0.4 in Constants, deploy, and drive a slow lap.

2. Set it back to 0.7, deploy, and drive the same lap.

3. Commit your working code, with a message that says what it does.

Done when: both laps are driven, and your working code is committed.

EXERCISE

Run the timed autonomous

1. Put the robot on blocks, wheels off the ground. Deploy.

2. In Driver Station, choose **Autonomous** and enable. Watch the wheels.

3. Set PAUSE_SECONDS to 2.0, deploy, and run it again.

Done when: the wheels drive forward, stop for two seconds, then turn, and the terminal prints Auto finished.

NEXT SEASON

What changes for 2027

In 2026	Changing for 2027
roboRIO	Systemcore: 2026 projects must be imported
edu.wpi.first packages	org.wpilib packages
Motor controller set(0.5)	setThrottle(0.5)
Java 17	Java 25
Commands v2	v2 remains, and v3 is new for Java

Alpha software, so this may change. Everything in this deck is correct for the 2026 roboRIO.

RECAP

What this deck covered

- What a subsystem, command, trigger, default command and the scheduler each do.
- CANDriveSubsystem: brushed motors, followers, one inverted side, configure().
- A gamepad binding with flipped signs, a deadband and a scale, tested with wheels up.
- Sequences built with andThen and Commands.sequence, and a timed autonomous.
- Bring-up symptoms, and open loop: why it works with a human driver.

Next: Deck 3: AdvantageScope shows what your robot is doing.

REFERENCE

Official docs



Command-based basics

docs.wpilib.org



Drive classes

docs.wpilib.org



SPARK MAX light patterns

docs.revrobotics.com



KitBot Java guide

FIRST KitBot resources